Studio Nx

JS 100 — initIntro(), initSideNavActiveState(), initMobileMenuClose()

2026-10-13

JS 100 — `initIntro()`, `initSideNavActiveState()`, `initMobileMenuClose()` (홈 화면 마무리 세 가지)

항목	내용
파일	`studionx/frontend/static/js/pages/home.js`, `home.html`
작업	P1-23 화면 마무리: 홈에 비어 있던 세 함수를 채움. 첫 접속 로딩을 처음에만 보이기, 세로 앵커의 현재 섹션 표시, 모바일 메뉴 닫기
작성	Claude (AI), 2026-10-13, 사용자 검토
이전 · 다음	98 `initDevNotesModal()` · 101 `privacy.js` 목차 표시와 정리 (다음)

작은 함수 세 개라서 한 문서에 함수별로 나눠 적었습니다.

A. `initIntro()` (첫 접속 로딩은 같은 탭에서 처음에만)

A-1. 하는 일

홈에 들어오면 약 1초짜리 로딩("Crescent")이 재생됩니다. 지금까지는 **홈에 올 때마다** 재생됐습니다. 이제 같은 탭에서 이미 봤다면 두 번째부터는 보이지 않습니다.

상황	결과
이 탭에서 처음	로딩이 재생되고, 탭에 `snx-intro = seen`을 기록
같은 탭에서 두 번째부터	로딩이 **전혀 보이지 않음**, 첫 화면 애니메이션은 원래 시간대로
새 탭이나 탭을 닫았다 열기	기록이 없으므로 다시 재생
저장소를 못 쓰는 환경(사생활 보호 모드 등)	오류 없이 열리고 매번 재생

A-2. 코드

`home.html`의 `<head>` (CSS가 적용되기 전에 실행되는 짧은 검사)

```
<script>try { if (sessionStorage.getItem('snx-intro') === 'seen') document.documentElement.dataset.intro = 'seen'; } catch (e) {}</script>
```

`home.js`

```
export function initIntro() {
  try {
    if (sessionStorage.getItem('snx-intro') === 'seen') {
      document.documentElement.dataset.intro = 'seen';
    } else {
      sessionStorage.setItem('snx-intro', 'seen');
    }
  } catch (err) {
    // 저장소를 못 쓰면 기록하지 못함 → 그냥 매번 보여 줌
  }
}
```

최신 문법으로 쓰면 (공부용)

```
export const initIntro = () => {
  try {
    if (sessionStorage.getItem('snx-intro') === 'seen') document.documentElement.dataset.intro = 'seen';
    else sessionStorage.setItem('snx-intro', 'seen');
  } catch {}
};
```

최신 문법	실제 코드	뜻
`catch {}`	`catch (err) { … }`	쓰지 않는 오류 변수를 생략합니다

식	뜻
`sessionStorage`	이 탭이 열려 있는 동안만 값을 기억하는 저장소입니다 (탭을 닫으면 사라짐)
`document.documentElement.dataset.intro = 'seen'`	`<html data-intro="seen">`을 답니다. `intro.css`가 이 속성이 있으면 로딩을 숨깁니다
`try … catch`	저장소가 막힌 환경에서는 접근만 해도 오류가 나므로 감쌉니다

A-3. 왜 이렇게

선택	이유
`<head>`에도 같은 검사를 한 줄 둠	`home.js`는 머리글과 바닥글을 끼운 뒤(서버에서 받아 옴)에야 실행되어 너무 늦습니다. 그때 숨기면 로딩이 잠깐 보였다가 사라지는 **깜빡임**이 생깁니다. `<head>`의 검사는 CSS가 적용되기 전에 실행되어 처음부터 숨깁니다 (시험 I05)
기록은 `initIntro`가 남김	기록하는 곳을 한 곳으로 합니다. `<head>`는 읽기만 합니다
`sessionStorage`(탭 단위)	"첫 접속"을 탭 단위로 봅니다. 홈의 로고를 눌러 다시 오거나 다른 페이지를 다녀와도 다시 보이지 않고, 새로 방문할 때만 봅니다
`initIntro`에도 읽는 검사가 있음	나중에 보안 정책으로 인라인 스크립트가 막히더라도, 깜빡임은 생기지만 로직은 동작하도록 하는 대비책입니다
저장소 접근을 `try`로 감쌈	사생활 보호 모드에서 `sessionStorage`가 오류를 내도 홈은 열려야 합니다

B. `initSideNavActiveState()` (세로 앵커에서 지금 보는 섹션 표시)

B-1. 하는 일

화면 오른쪽의 세로 앵커(`S`, `Ch`, `F`, `D`, `Co`)에서, **지금 화면 가운데에 있는 섹션**의 링크에 `aria-current="true"`를 달아 보라색으로 표시합니다 (CSS는 이미 있었습니다).

스크롤 위치	표시되는 링크
맨 위의 첫 화면(hero)	없음
Service, Character, Feature, Development, Contact 섹션이 화면 가운데에 옴	그 섹션의 링크 하나만
맨 아래	마지막 섹션(Contact)
두 섹션의 경계	하나 이하 (위쪽 것)

B-2. 코드

```
export function initSideNavActiveState() {
  const links = document.querySelectorAll('.side-nav__link');
  if (links.length === 0 || typeof IntersectionObserver === 'undefined') return;

  const targets = [];   // { id, link, section, inside }
  for (const link of links) {
    const href = link.getAttribute('href') || '';
    if (href.charAt(0) !== '#') continue;
    const section = document.getElementById(href.slice(1));
    if (section) targets.push({ id: href.slice(1), link: link, section: section, inside: false });
  }
  if (targets.length === 0) return;

  const refresh = function () {
    let current = null;
    for (const target of targets) {
      if (target.inside) { current = target; break; }
    }
    for (const target of targets) {
      if (target === current) target.link.setAttribute('aria-current', 'true');
      else target.link.removeAttribute('aria-current');
    }
  };

  const observer = new IntersectionObserver(function (entries) {
    for (const entry of entries) {
      for (const target of targets) {
        if (target.section === entry.target) target.inside = entry.isIntersecting;
      }
    }
    refresh();
  }, { rootMargin: '-50% 0px -50% 0px', threshold: 0 });

  for (const target of targets) observer.observe(target.section);
}
```

최신 문법으로 쓰면 (공부용)

```
export const initSideNavActiveState = () => {
  const targets = [...document.querySelectorAll('.side-nav__link')]
    .map((link) => ({ link, section: document.getElementById(link.getAttribute('href')?.slice(1)), insid
e: false }))
    .filter((t) => t.section);
  if (!targets.length || !('IntersectionObserver' in window)) return;
  const observer = new IntersectionObserver((entries) => {
    entries.forEach(({ target: el, isIntersecting }) => { targets.find((t) => t.section === el).inside =
isIntersecting; });
    const current = targets.find((t) => t.inside);
    targets.forEach((t) => t.link.toggleAttribute('aria-current', t === current));
  }, { rootMargin: '-50% 0px -50% 0px' });
  targets.forEach((t) => observer.observe(t.section));
};
```

최신 문법	실제 코드	뜻
`?.slice(1)`	`	
`.map(...).filter(...)`	`for` 반복과 `if`	목록을 바꾸고 거릅니다
`{ link, section }`	`{ link: link, section: section }`	이름이 같으면 줄여 씁니다
`toggleAttribute`	`setAttribute`와 `removeAttribute`	속성을 켜고 끕니다

식	뜻
`IntersectionObserver`	요소가 화면의 어느 범위에 걸치는지를 알려 주는 브라우저 기능입니다. 스크롤 이벤트를 계속 계산하지 않아 가볍습니다
`rootMargin: '-50% 0px -50% 0px'`	관찰 범위를 위아래 50%씩 잘라 내서 **화면 가운데의 얇은 띠**로 만듭니다. 섹션이 이 띠에 걸치면 "지금 보는 섹션"입니다
`entry.isIntersecting`	그 섹션이 띠에 걸쳐 있는지입니다
`aria-current="true"`	"지금 위치"라는 뜻입니다. 화면 낭독기도 읽고, CSS가 이 속성으로 보라색을 칠합니다
`href.slice(1)`	`#service`에서 `#`를 뗀 `service`입니다

B-3. 왜 이렇게

선택	이유
화면 **가운데 띠** 기준	섹션이 화면 한 장을 꽉 채우므로(`min-height: 100svh`) 가운데에 오는 것이 "보고 있는 것"입니다. 위쪽 기준이면 다음 섹션이 보이기 시작하자마자 바뀝니다
섹션마다 `inside`를 기억하고 표시는 한 번에 계산(`refresh`)	스크롤하면 한 섹션이 나가고 다음이 들어오는 두 변화가 따로 올 수 있습니다. 그때 중간에 두 개가 표시되거나 모두 사라지는 순간이 생기지 않게, 변화를 모두 반영한 뒤 한 번에 정합니다
첫 화면과 섹션 사이에서는 표시 없음	아무 섹션도 아닌 곳에 표시가 남아 있으면 잘못된 안내입니다
링크의 `href`에서 섹션을 찾음	섹션을 추가해도 링크만 더하면 됩니다. 섹션이 없는 링크는 건너뜁니다
`IntersectionObserver`가 없으면 그냥 끝냄	표시만 빠질 뿐 링크 이동은 그대로 됩니다
처음에도 표시가 맞음	관찰을 시작하면 현재 상태가 곧바로 한 번 알려집니다. 예를 들어 `#feature` 주소로 들어와도 맞게 표시됩니다

C. `initMobileMenuClose()` (모바일 메뉴에서 링크를 누르면 닫기)

C-1. 하는 일

폭이 좁은 화면(모바일, 태블릿)에서는 머리글에 `Menu` 버튼이 있고, 누르면 섹션 링크 5개와 로그인 버튼들이 든 상자가 열립니다. 홈에서 링크를 누르면 **그 섹션으로 이동하지만 메뉴가 열린 채로 남아** 화면을 가렸습니다. 이제 메뉴 안의 링크나 버튼을 누르면 메뉴가 닫힙니다.

누른 곳	결과
메뉴 안의 링크 (`Service`, `Character` 등)	메뉴가 닫히고 그 섹션으로 이동
메뉴 안의 버튼 (`Sign In`)	메뉴가 닫히고 로그인 창이 열림
메뉴 안의 링크도 버튼도 아닌 곳	닫히지 않음

C-2. 코드

```
export function initMobileMenuClose() {
  const menus = document.querySelectorAll('details.menu');
  for (const menu of menus) {
    const panel = menu.querySelector('.menu__panel');
    if (!panel) continue;
    panel.onclick = function (event) {
      if (event.target.closest('a, button')) menu.open = false;
    };
  }
}
```

최신 문법으로 쓰면 (공부용)

```
export const initMobileMenuClose = () => {
  document.querySelectorAll('details.menu').forEach((menu) => {
    menu.querySelector('.menu__panel')?.addEventListener('click', ({ target }) => {
      if (target.closest('a, button')) menu.open = false;
    });
  });
};
```

최신 문법	실제 코드	뜻
`?.addEventListener`	`if (!panel) continue;`	요소가 없으면 건너뜁니다
`.forEach((menu) => …)`	`for (const menu of menus)`	목록을 하나씩 처리합니다
`({ target }) =>`	`function (event)` 와 `event.target`	화살표 함수와 구조 분해입니다

식	뜻
`<details class="menu">`	펼치고 접는 HTML 요소입니다. `open` 속성이 있으면 펼쳐진 상태입니다
`menu.open = false`	메뉴를 닫습니다
`event.target.closest('a, button')`	누른 곳에서 위로 올라가며 가장 가까운 링크나 버튼을 찾습니다. 링크 안의 글자를 눌러도 링크를 찾습니다
`panel.onclick = …`	처리기를 하나만 둡니다. 두 번 불러도 겹치지 않고 덮어씁니다

C-3. 왜 이렇게

선택	이유
링크뿐 아니라 **버튼** (Sign In)도 닫음	로그인 창이 메뉴 위에 겹쳐 뜨면 창을 닫은 뒤 메뉴가 그대로 남아 어색합니다
빈 곳을 누르면 닫지 않음	메뉴 안의 여백을 잘못 눌렀다고 닫히면 불편합니다
메뉴 상자 (`.menu__panel`)에 하나만 연결	링크와 버튼이 몇 개든 한 곳에서 받습니다
`includePartials()` 뒤에 호출	메뉴는 공통 머리글 조각이라 서버에서 받아 끼운 뒤에야 존재합니다. 이 파일 맨 아래의 호출 순서가 이미 그렇습니다
홈에서만 연결	다른 페이지의 메뉴 링크는 `home.html#…`로 **페이지를 이동**해서 메뉴가 저절로 사라집니다. 같은 문제가 나는 곳은 홈뿐입니다 (다른 페이지의 Sign In 버튼은 로그인 모달이 홈에만 있어 해당 없음)

시험 (실제 서버, 헤드리스 Chrome, 마우스와 키보드로 조작, 29개)

`initIntro` (9개)

#	한 일	결과
I01	새 탭에서 처음 홈을 엶	로딩이 보임 ✓
I02	1.8초 뒤	탭에 `snx-intro = seen`이 기록됨 ✓
I03	로딩의 끝	1초 안팎에 사라짐 ✓
I04	같은 탭에서 홈을 다시 엶	로딩이 보이지 않음 ✓
I05	`DOMContentLoaded` 시점(`home.js`가 헤더를 끼우기 전)	이미 `data-intro="seen"`이고 로딩은 `display: none`, 즉 깜빡임 없음 ✓
I06	두 번째 방문	`data-intro="seen"` 유지 ✓
I07	기록을 지우고 다시 엶 (새 탭과 같음)	로딩이 다시 보임 ✓
I08	저장소가 막힌 환경	오류 없이 열리고 공지 카드까지 채워짐 ✓
I09	저장소가 막힌 채로 다시 엶	매번 로딩이 보임 ✓

`initSideNavActiveState` (13개)

#	한 일	결과
N01	맨 위(첫 화면)	표시 없음 ✓
N02 ~ N06	Service, Character, Feature, Development, Contact 섹션을 차례로 화면 가운데로	각각 그 링크 **하나만** `aria-current` ✓
N07	맨 아래	Contact 유지 ✓
N08	다시 맨 위	표시가 모두 사라짐 ✓
N09	두 섹션의 경계	표시는 하나 이하 ✓
N10	세로 앵커의 `F`를 마우스로 누름	Feature로 이동(부드러운 스크롤 뒤)하고 `F`가 표시됨 ✓
N11	함수를 두 번 더 부름	표시는 하나 이하, 오류 없음 ✓
N12	가리키는 섹션이 없는 링크만 있음	오류 없이 끝남 ✓
N13	`IntersectionObserver`가 없음	오류 없이 끝남 ✓

`initMobileMenuClose` (7개, 폭 390)

#	한 일	결과
B01	`Menu`를 마우스로 누름	메뉴가 열림 ✓
B02	메뉴의 `Character` 링크를 누름	메뉴가 닫히고 `#character`로 이동 ✓
B03	메뉴 상자의 링크도 버튼도 아닌 곳을 누름	닫히지 않음 ✓
B04	메뉴의 `Sign In` 버튼을 누름	메뉴가 닫히고 로그인 창이 열림 ✓
B05	다시 열어서 `Contact` 링크를 누름	닫히고 `#contact`로 이동 ✓
B06	함수를 두 번 더 부르고 다시 시험	정상 동작 ✓
B07	메뉴 요소가 없음	오류 없이 끝남 ✓

29개 모두 통과했습니다. 시험을 만드는 중에 시험 쪽 문제가 몇 번 있었습니다. B03, B04는 직전 단계에서 페이지가 스크롤된 채라 `Menu` 버튼이 헤더에 가려 눌리지 않았던 것이라, 단계마다 홈을 새로 열도록 고쳤습니다. 한 번은 로딩 시험(I01)이 서버가 막 켜진 직후 250ms 안에 로딩을 못 본 적이 있었고, 이후 같은 시험을 세 번 더 돌려 모두 통과했습니다 (시간 여유가 빠듯한 시험입니다). 앱 코드를 고친 곳은 없습니다.

개정 (2026-10-13, P1-25): 맨 위로 앵커

세로 앵커 맨 위에 `#top`(첫 화면)으로 가는 화살표 링크를 더했습니다(66 §3-2). `initSideNavActiveState()`는 이 링크를 구간 표시 대상에서 뺍니다.

```
if (href.charAt(0) !== '#' || href === '#top') continue;   // '#top'(맨 위로)은 구간이 아니라 이동 버튼
```

맨 위로 올라가면 첫 화면(`#top`)이 감지 띠에 걸려서 화살표가 구간처럼 보라색으로 표시되었기 때문입니다. 최신 문법으로 쓰면 `if (!href.startsWith('#') || href === '#top') return;` 입니다.